



COMP 4300

Intro to Game Programming

Lecture 17
Game Performance
Memory Pooling
CPU Caching

Game Performance

- There are **many factors** that determine the overall performance of our game
- With ECS, we can profile our systems **independently** and determine which are bottlenecking the program's speed
- Typical bottlenecks are Physics, AI, Rendering, Ray Casting, etc

Process: [9696] Demo.RayTracer.exe

Just Trace Session - Program.cs

Call Tree

Back Forward Get Compare Show Timeline Kill Profile Merge Signatures Average Times Merge Minor Calls New Merge Caller Find in Expand Source

NAVIGATION TIMELINE CONTROL CALL TREE SELECTED CALL

Current views: Snapshot for 09:27:21 - 09:27:22

OVERVIEW Metrics Assemblies CALL TREES All Threads METHODS All Methods By Namespace ASP.NET Requests Outgoing Web Requests SQL Queries CALLER TREES Hot Spots Bottlenecks

Search:

Name	Own Time (ms)	Total Time (ms)	Hit Count
Process # 9696 (Demo.RayTracer)		45,078 100.00%	
System.Threading.ExecutionContext.Run(System.Threading.ExecutionContext executionContext, System.Threading.ContextCallback callback, object state)	61 0.14%	22,318 49.51%	2
Demo.RayTracer.ThreadData+<>c__DisplayClass2.<.ctor>b_0(object o)	2 -	22,257 49.37%	2
Demo.RayTracer.RayTracer+<>c__DisplayClass4.<StartRender>b_2(RayTracing.RenderParameters o)	4 0.01%	22,255 49.37%	2
RayTracing.RayTracer.Render()	1,677 3.72%	21,849 48.47%	2
RayTracing.RayTracer.TraceRay(RayTracing.Ray ray, RayTracing.Scene scene, int depth)	73 0.16%	18,993 42.13%	88,804
RayTracing.RayTracer.Shade(RayTracing.ISect isect, RayTracing.Scene scene, int depth)	163 0.36%	17,573 38.98%	75,692
RayTracing.RayTracer.GetReflectionColor(RayTracing.SceneObject thing, RayTracing.Vector pos, RayTracing.Vector norm, RayTracing.Vector rd, RayTracing	73 0.16%	9,088 20.16%	75,692
RayTracing.RayTracer.TraceRay(RayTracing.Ray ray, RayTracing.Scene scene, int depth)	53 0.12%	8,900 19.74%	75,692
RayTracing.RayTracer.Shade(RayTracing.ISect isect, RayTracing.Scene scene, int depth)	69 0.15%	7,809 17.32%	30,060
RayTracing.RayTracer.GetReflectionColor(RayTracing.SceneObject thing, RayTracing.Vector pos, RayTracing.Vector norm, RayTracing.Vector rd, Ra	23 0.05%	4,338 9.62%	30,060
RayTracing.RayTracer.GetNaturalColor(RayTracing.SceneObject thing, RayTracing.Vector pos, RayTracing.Vector norm, RayTracing.Vector rd, RayTr	185 0.41%	3,128 6.94%	30,060
6 minor calls grouped	142 0.31%	274 0.61%	263,145

105 3 (0.01%) 300 for (int y = parameters.RenderRectangle.Top; y < parameters.RenderRectangle.Bottom && !parameters.AbortSignaled; y += parameters.RenderStep)

106 0 (-) 298 {

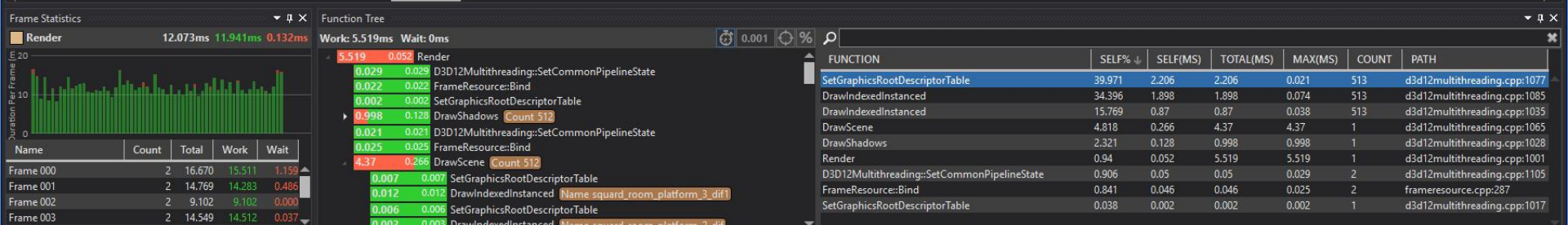
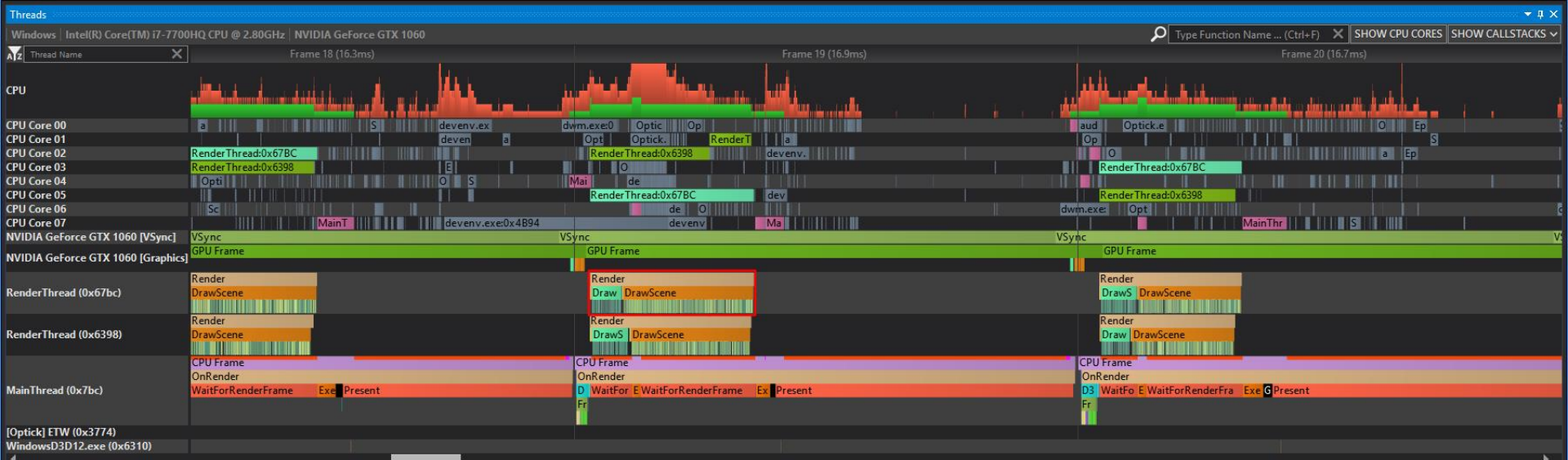
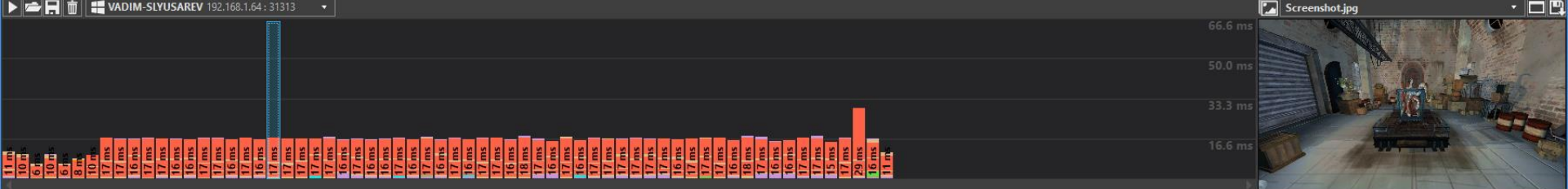
Namespace: RayTracing
Class Name: RayTracer
Method: RayTracing.RayTracer.Render()

Help

Call Stack Breakpoints Command Window Output

No errors in solution | Ready

Solution Explorer Team Explorer



C++ Profiling Videos

- Visual Benchmarking in C++
Manual Tools for Profiling / Visualization
<https://www.youtube.com/watch?v=xIAH4dbMVnU>
- Debugger/Diagnostics in VS 2019
Tips/Tricks for VS2019
<https://www.youtube.com/watch?v=i6gdmT-BdOU>
- Tuning C++: Benchmarks, CPUs, Compilers
<https://www.youtube.com/watch?v=nXaxk27zwlk>
- More in the next lecture!

Non-Obvious Performance Hits

- It's easy to think that the only things that affect performance are **algorithms / math**
- This is somewhat true – changing an n^2 algorithm to $\log(n)$ can save lots of time
- However, in practice, often times the **implementation** of these algorithms can impact performance even more

Important Optimizations

- Using more **efficient** algorithms
 - This is where computer science helps
 - But also where computer science stops
- Minimize **size** of data structures
- Code re-use and modularity
- Minimize **dynamic** heap allocations
 - Memory Pooling
- Write **cache-friendly** code
 - Data-oriented design/programming

Minimizing Data Structure Size

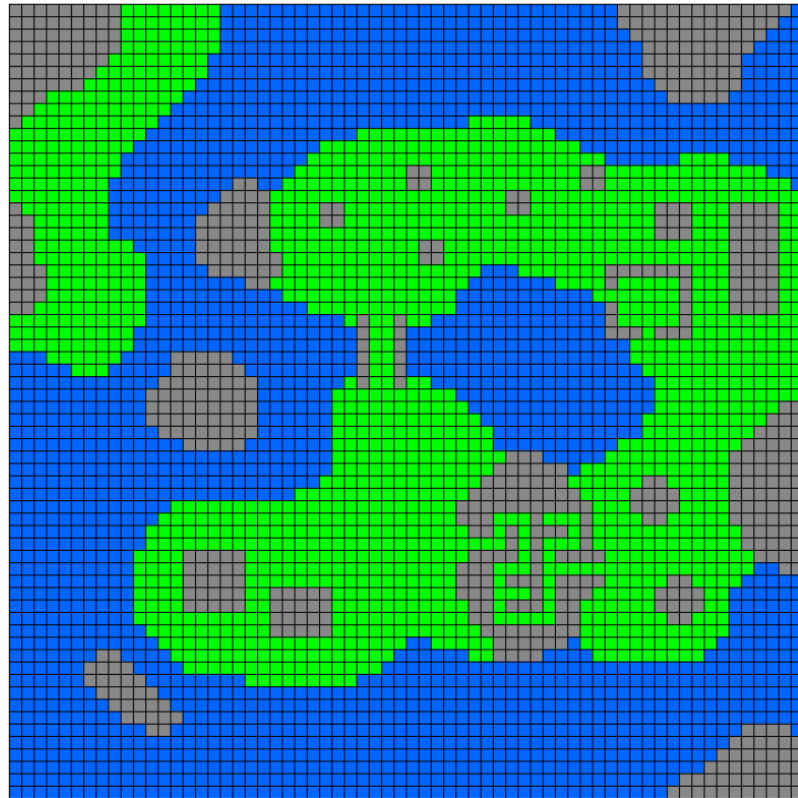
- C++ gives us **fine control** over the size of data structures we can use for games
- Deciding on a size type for your data depends entirely on the **context** for your game, function, structure, etc
- **Minimizing** data structure size can help your game in several ways

Minimizing Data Structure Size

1. Reducing size = use **less memory**
Obvious benefit: less ram usage
2. Reduced size = **less time copying** data
If data needs to be copied in your program
this can dramatically time (ex: vector)
3. Size < 8 bytes = pass by value
Eliminates dereference on function call /
cache miss on function calls

Data Size Example

- Game world size 64x64
- Don't need full int to store player location
- uchar = 256 values
- But maybe later, map will be bigger? Careful
- Optimize once finalized



Code Reuse and Modularity

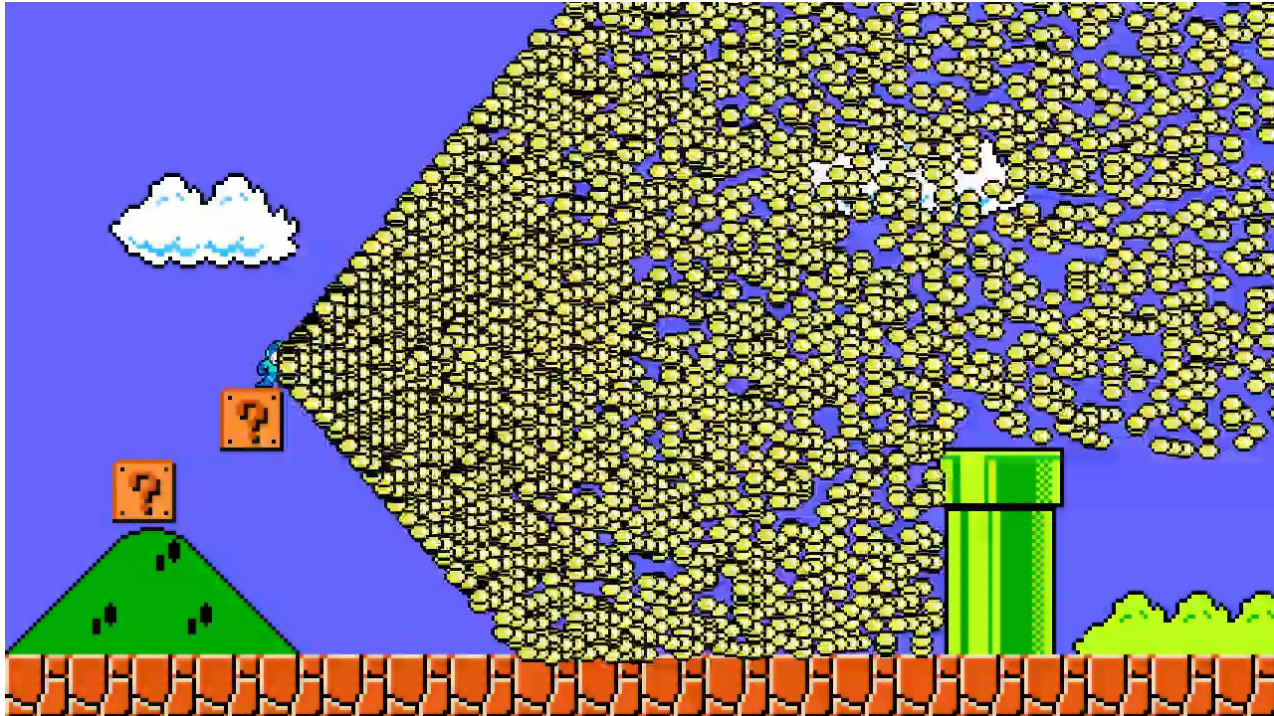
- We have talked about this before
- Put any re-used code into its **own function**
- Optimize the function in **one place**
- EntityManager – We use this all the time without having to know **how** it works
- Any optimizations underneath are invisible and **don't require top level changes**

Minimize Dynamic Allocations

- Dynamic **heap allocations** are expensive
- Call to `malloc()`, OS dependent
- Tries to find a **contiguous block of memory** of a given size
- Given the complexities of memory layout, is **much much slower** than stack alloc



Entity Creation / Deletion



Memory Pooling

- Heap memory is **slower** to allocate
- Stack memory is too **limited** (few MB)
- What we could do: Allocate a large chunk of heap memory at the start of the program and use only that space (**memory pooling**)
- Eliminates allocations **during runtime**
- However, adds the complexity of **managing the memory pool** and where to put new objects

Cache Memory

Cache Coherency

- Writing **cache-friendly** code is one of the most important speed optimizations in practice
- When data is fetched from RAM, **surrounding values** also fetched and stored in cache (~64b typical)
- Cache memory is **faster**, physically closer, but is much smaller than normal RAM
- Organizing frequently used data **contiguously** allows cache to fetch more of it each lookup
- When RAM is not in cache = 'cache miss'

Memory Lookup Procedure

- When you access data in code, the following sequence occurs:
 1. Look up data in L1 cache
 2. Look up data in L2 cache
 3. Look up data in L3 cache
 4. Look up data in DRAM

CPU

Core

L1

L2

Core

L1

L2

Core

L1

L2

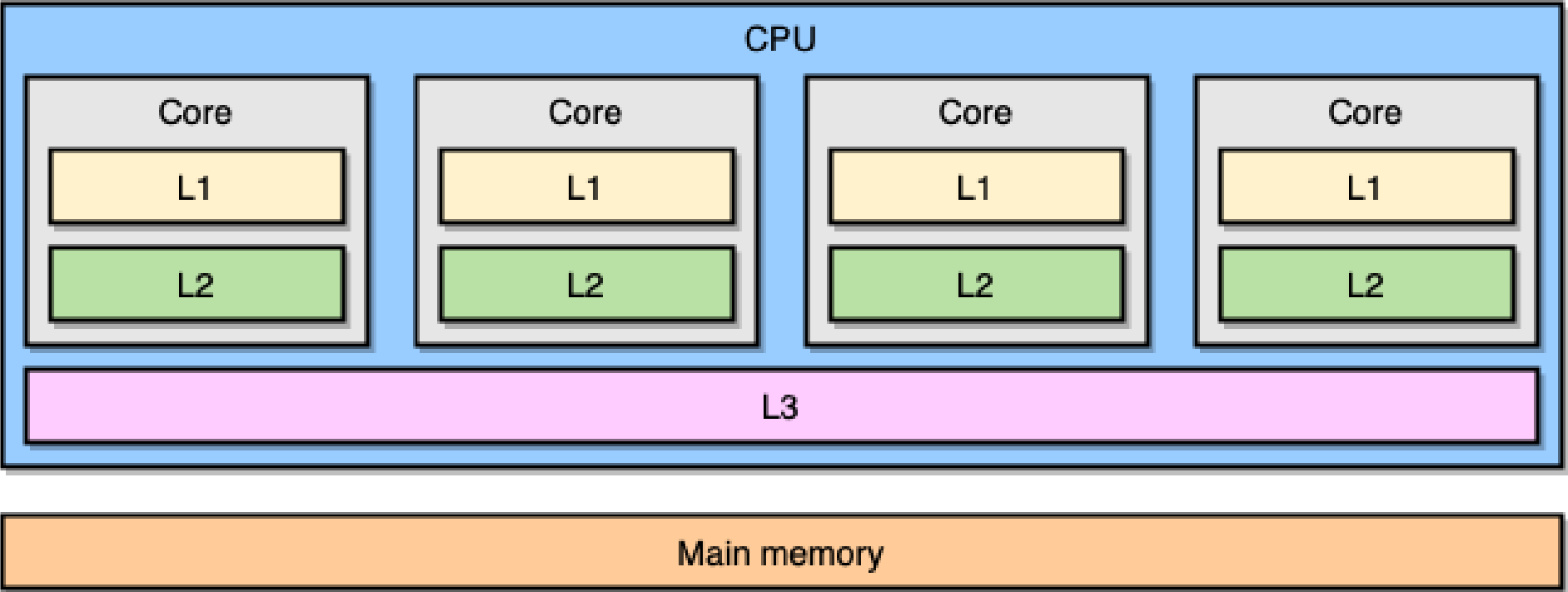
Core

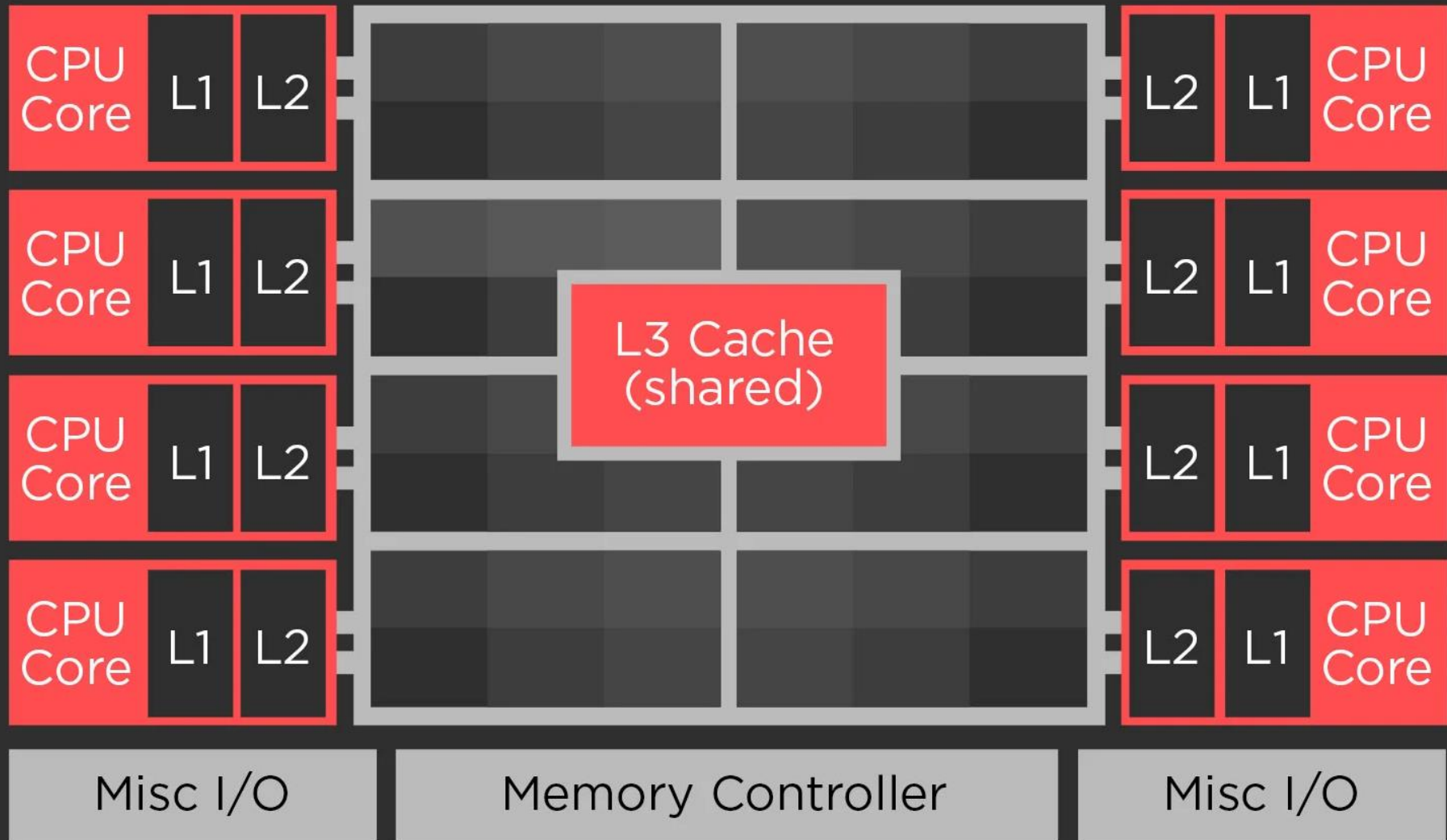
L1

L2

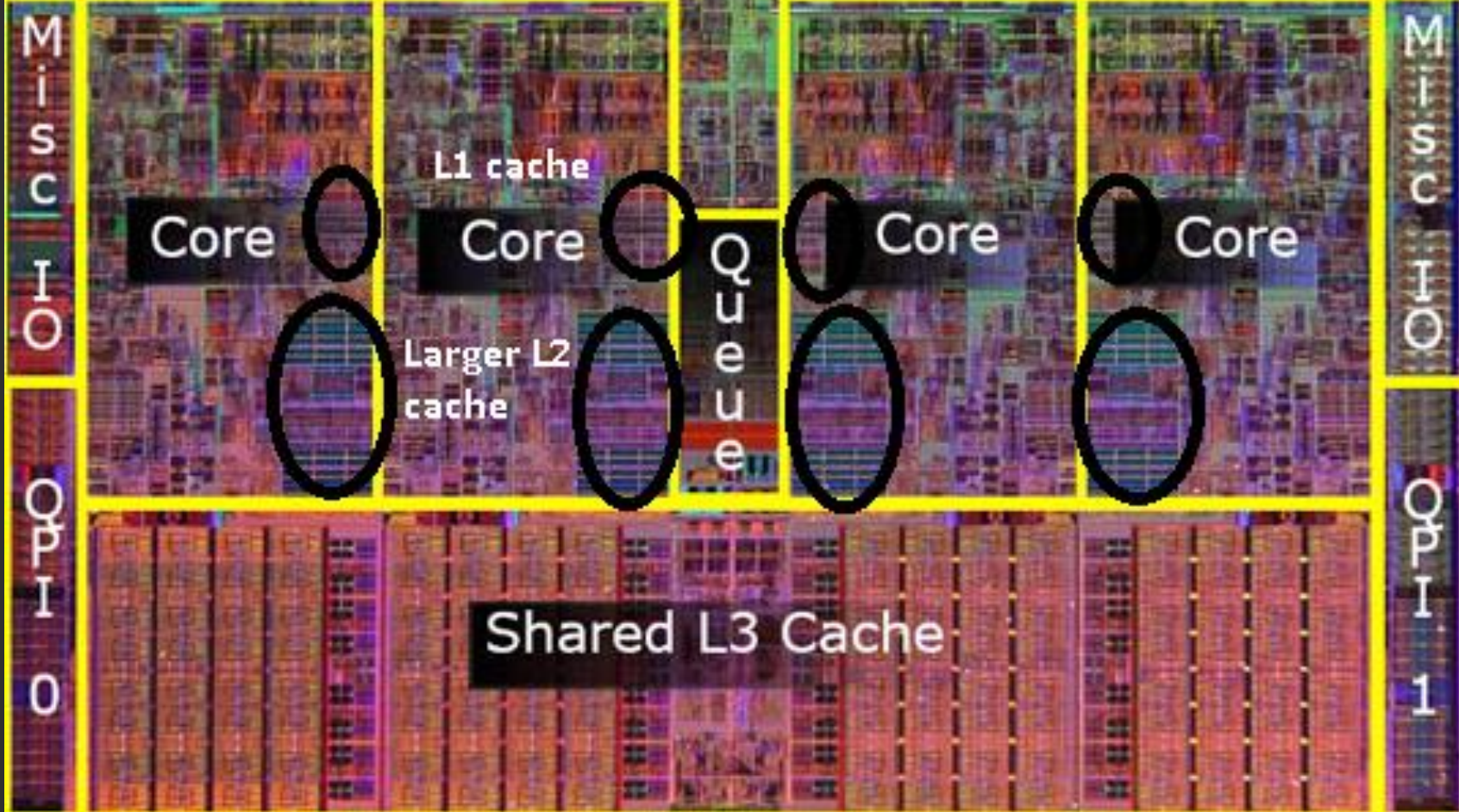
L3

Main memory

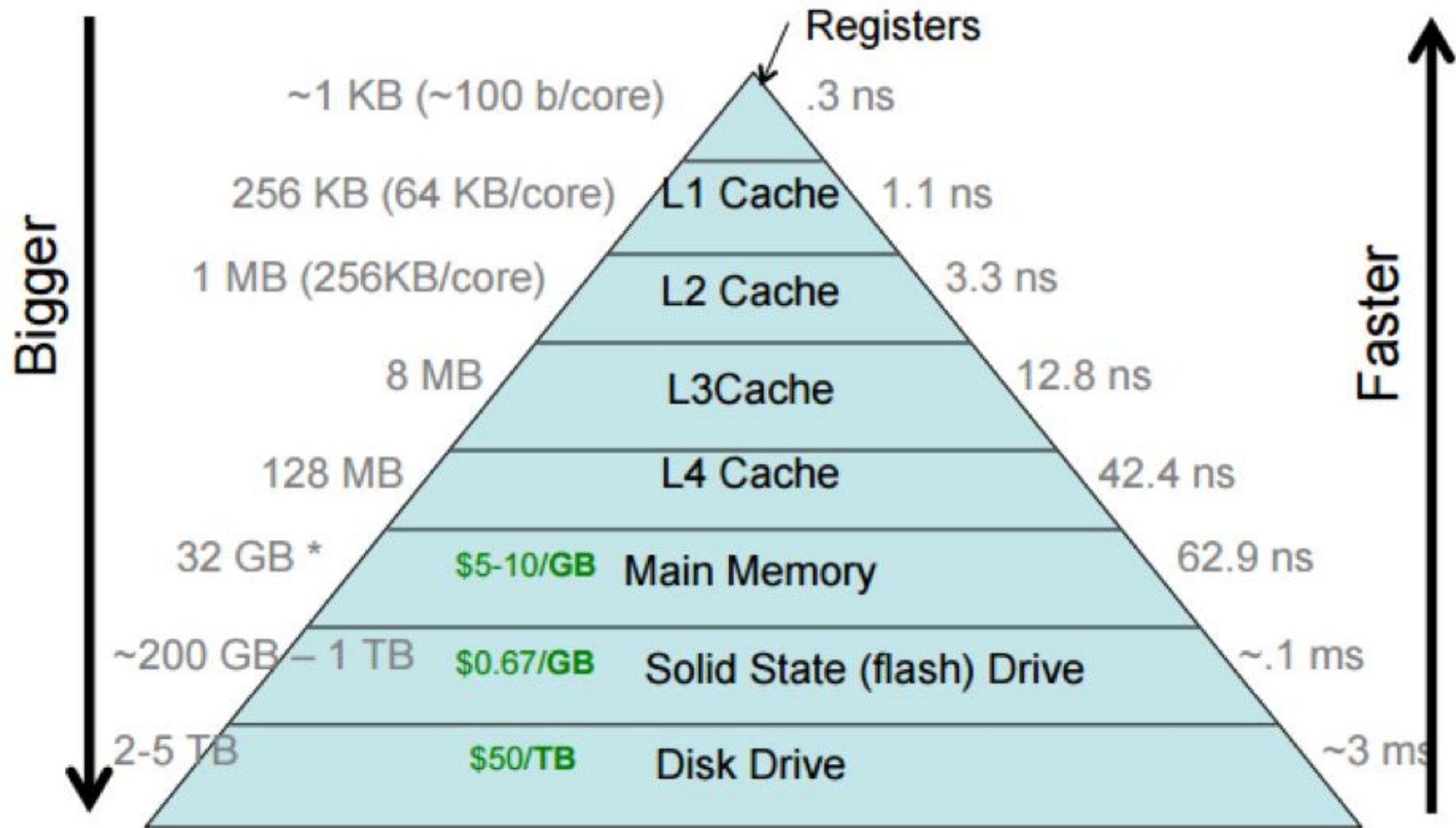




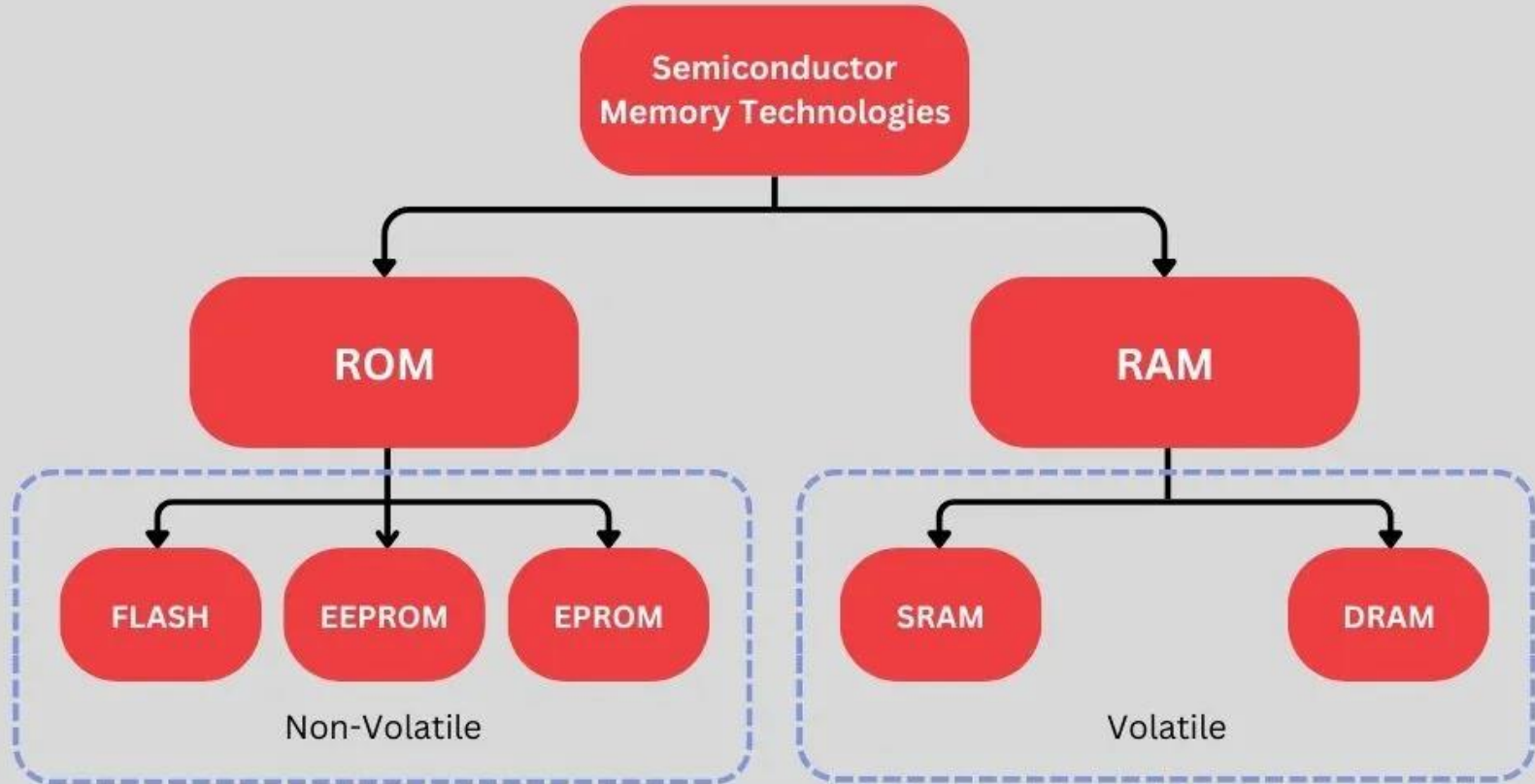
Memory Controller



	Latency	Bandwidth / Channel	Max Capacity*	Significance	Programmers View
Reg	0.2ns		KB	In CPU	L1 - dereference pointer
Cache	40ns		KB		
DDR (Main)	80-140ns	32-51.2 GB/s (DDR5)	Up to 4TB		L2 - dereference pointer high perf memcpy
DDR (NUMA)	170-250ns	32-51.2 GB/s (DDR5)	Up to 8TB		
DDR (CXL)	170-250ns	32-51.2 GB/s (DDR5)	2-4 TB	CPU independent but local	L3 - dereference pointer high perf memcpy, swap
DDR (CXL Switched)	300-400ns	32-51.2 GB/s (DDR5)	64TB		
Far Memory	2-4us	100 GB/s (800g ethernet)	infinite	Network attached	L4 - memcpy, swap
SSD	50-100us				L5 - memcpy, swap



Embedded Systems Memory Types



DRAM vs SRAM

- DRAM: **Dynamic** Random Access Memory
- Used in main **system** memory (RAM)
- Stores data in **capacitors**, which require periodic refreshing (more power)
- **Lower** cost per byte (cheaper to make)
- **Slower** than SRAM

DRAM vs SRAM

- SRAM: **Static** Random Access Memory
- Used in CPU **cache** memory
- Stores data in **flip flops**, which do not need refreshing like DRAM
- **Higher** cost per byte
- **Faster** than DRAM
- Used in conjunction with DRAM for balance

Cache Misses

- Cache misses are expensive
 - Note: Example Calculations (not exact)
- Cache miss $\sim$ = 200 cycles
- Multiplication = 3 cycles
- You can do 65 multiplications in the same time it takes you to look up an variable value in RAM if it's not cached

Example 1: Memory accesses and performance

How much faster do you expect Loop 2 to run, compared Loop 1?

```
int[] arr = new int[64 * 1024 * 1024];
```

```
// Loop 1
```

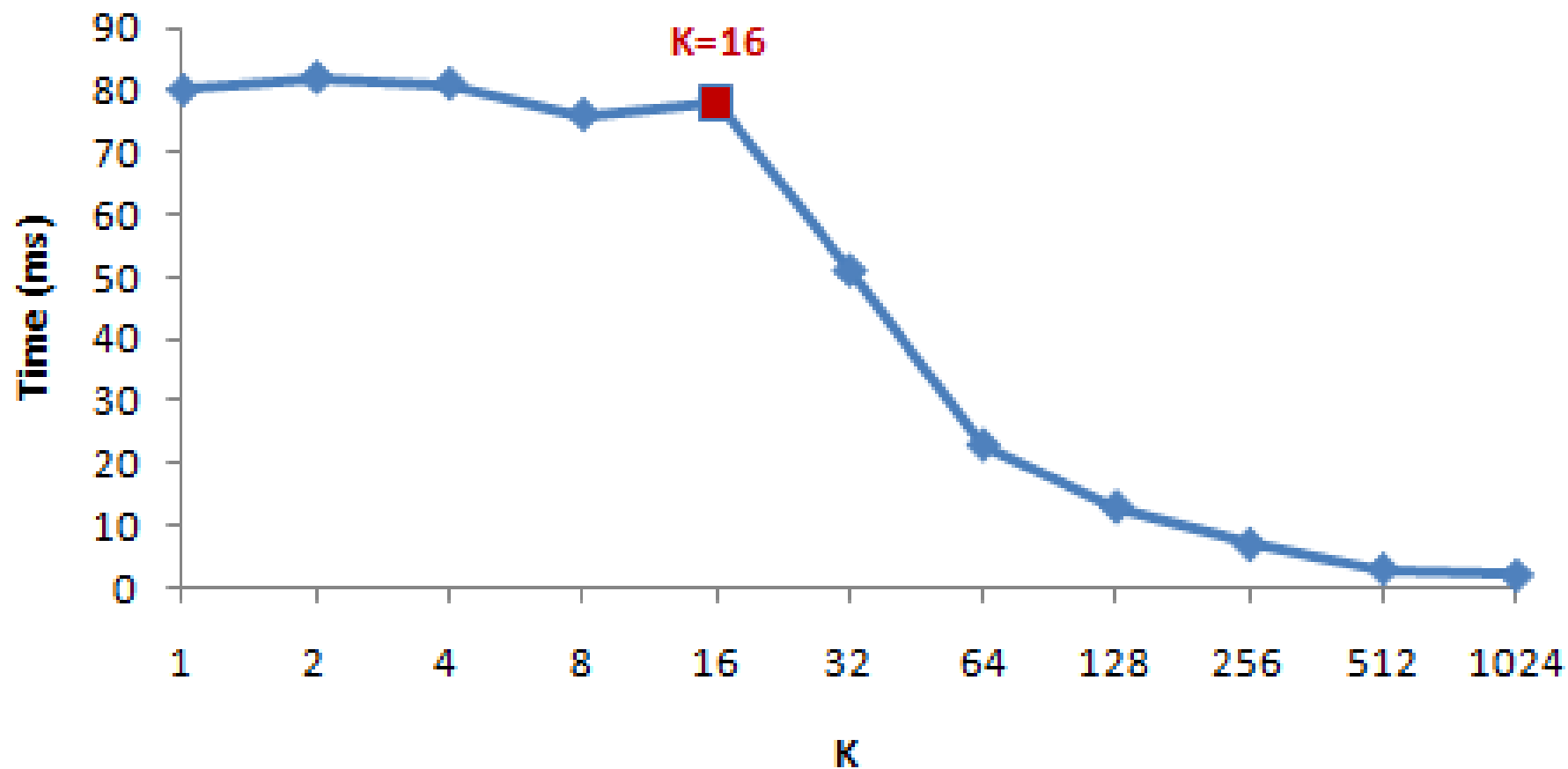
```
for (int i = 0; i < arr.Length; i++) arr[i] *= 3;
```

```
// Loop 2
```

```
for (int i = 0; i < arr.Length; i += 16) arr[i] *= 3;
```

Loop 2 does 6% of the CPU 'work', but takes same time

Update Every K-th Int



Example 1 Explained

- Memory **access** dominates running time
- Ex: Cache will grab **64b = 16 ints**
- Loop 1 has 1 normal lookup and 15 cached lookups (almost free)
- Loop 2 has 1 normal lookup, but $1/16^{\text{th}}$ of them. Similar memory access time
- Multiplications are pipelined, **negligible**

More Cache Examples

- Gallery of Processor Cache Effects
<http://igoro.com/archive/gallery-of-processor-cache-effects/>
- Data-Oriented Design and C++
Writing cache-friendly C++ code
<https://www.youtube.com/watch?v=rX0ItVEVjHc>

Cache Friendly Data Structures

- In order to avoid cache misses, we want to **store data contiguously** in memory
- **Arrays and vectors** do this, but certain lists, sets, maps, dictionaries do not
- Linked lists often use pointers to heap allocated memory, which is very slow
- Use vector / array where possible

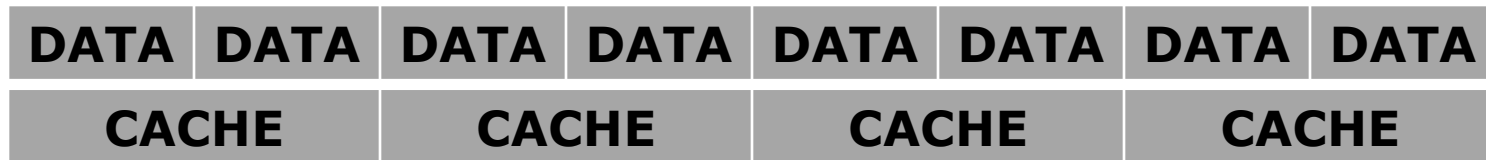
Vector Note

- A vector is a **dynamically** sized array
- When a vector is filled, it will create a new, larger temp array and **copy everything** over
- To avoid vector resize, we can **reserve** an amount of space equal to some **maximum** size
 - `vec.reserve(MaxElements)`
 - Uses a single malloc, all ram used immediately
 - Inserts into vector won't resize until $> \text{MaxElements}$

Vector Storage Options

- `vector<Data>`
 - Storing raw data in vec makes it contiguous
 - Raw data is larger, more expensive to copy
- `vector<Data *> vec;`
 - Storing pointers usually means **non-contiguous** data
 - Pointers are easier to copy, but **not cache friendly** (unless what it points to is intentionally contiguous [pooled])
 - If `std::shared_ptr`, everything will cache miss
- Right now, EntityManager has a vec of pointers
- Every time we get entity -> **cache miss**

Vector: Store Data vs Pointer



Contiguous data allows cache to fetch multiple



Pointers cached, **not the data they point to**
Each dereference will likely be a cache miss
Unless the data it points to is contiguous

ECS and Caching

- Entities are **containers** for components
- Entities currently contain all component data in a `std::tuple`, which is contiguous
- If all of the Entity components fit within the cache line, **the entire entity will be cached** for use
- However, is this what we want?

Entity
Component1
Component2
Component3
Component4

Entities and Caching

E1	E2	E3	E4	E5	E6	E7	E8
CACHE		CACHE		CACHE		CACHE	

If Entities stored in Vector, they are contiguous
Cache could load multiple entities
Currently, we store `shared_ptr<Entity>`
But let's pretend that we are doing it this way

Entities and Caching



Entities stored contiguously
Components of same type are not
Why does this matter?

ECS Systems and Caching

- Storing Entities contiguously makes sense, since the cache can now fetch multiple
- However, caching is all about **ACCESS**
- **How we want to access the data** inside Entities determines how we should store
- Let's look at some examples of the systems we use in our game engine

ECS Systems Example

```
// movement
for (auto e : entities)
    e.get<Transform>.pos += e.get<Transform>.velocity
// rendering
for (auto e : entities)
    draw(e.get<Animation>.sprite)
// collisions
for (auto e : entities)
    // e.get<CBoundingBox>
    // e.get<Transform>
```

ECS Systems Example

```
// movement
for (auto e : entities)
    e.get<Transform>.pos += e.get<Transform>.velocity

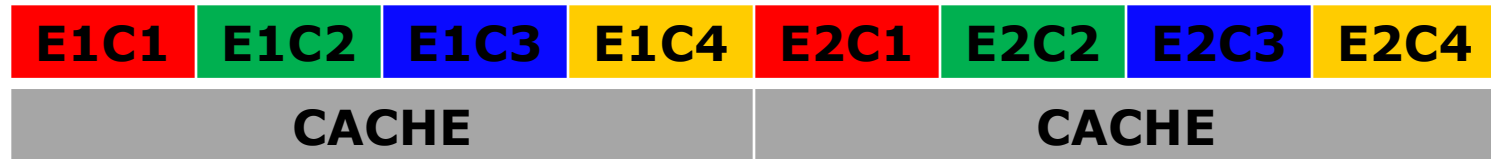
// rendering
for (auto e : entities)
    draw(e.get<Animation>.sprite)

// collisions
for (auto e : entities)
    // e.get<CBoundingBox>
    // e.get<Transform>
```

ECS Systems and Caching

- ECS systems usually only access **one or two** different types of components for each Entity that they operate on
- Given this access pattern, we would want the cache to **store similar components** rather than entire entities
- Components are **small**, so the cache line **can fit many more** than full Entities

Entities and Caching



This is what we have seen so far

Entities and Caching

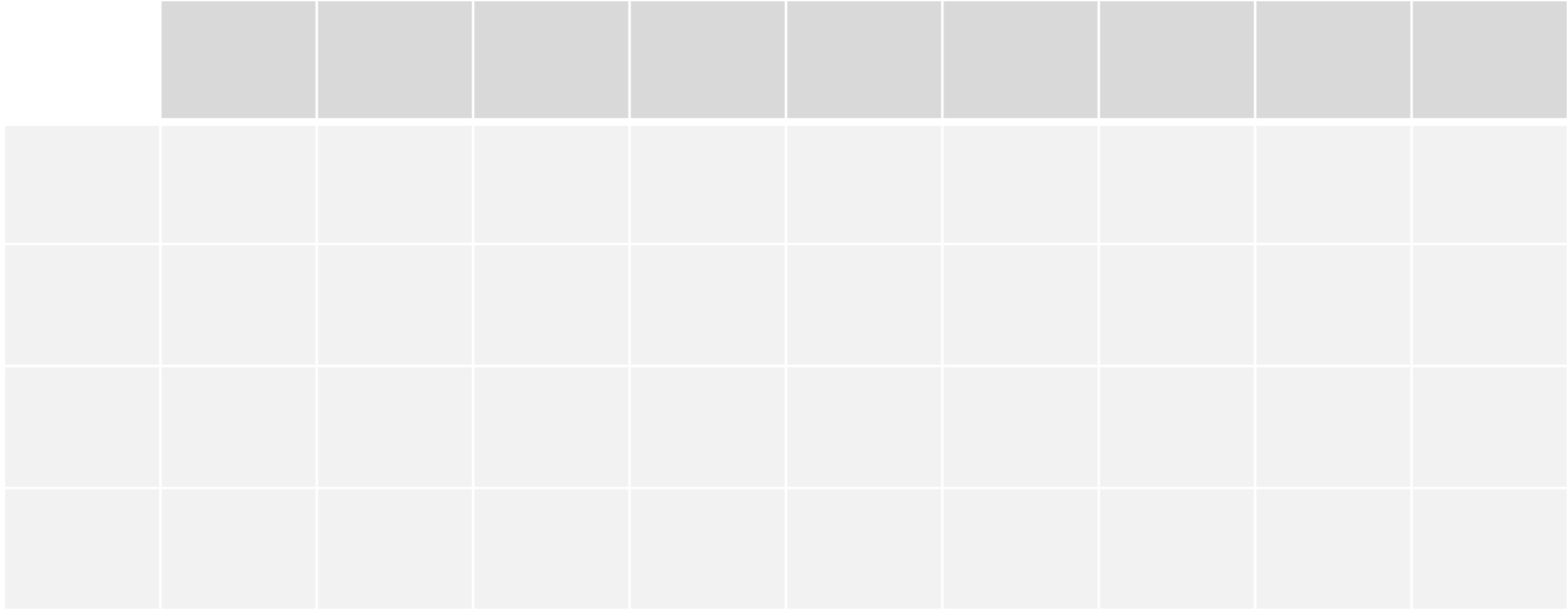


Ideally, this is what we want to have.
ECS systems operate on similar components.
Components contiguous = more cache hits.
How can we achieve this?

ECS Memory Pool

- A memory pool is a **pre-allocated** chunk of memory that will be used for the storage of data of a given type (ex: Entities)
- By pre-allocating space, we **don't need to use dynamic memory** allocation
- May need to know **max size** before hand

ECS Memory Pool



ECS Memory Pool - Vec<Entity>

	E0	E1							
C1									
C2									
C3									
C4									

ECS Memory Pool - Vec<Entity>

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1									
C2									
C3									
C4									

ECS Memory Pool - Vec<Entity>

- This memory pool can **eliminate all runtime dynamic memory allocations** by being big enough to fit all the entities
- We **trade** upfront memory for runtime speed
- When new entity created, we need to find an 'empty spot' for it in the table (memory pool)
- This is much faster than malloc in practice
 - Ex: Scan left to right for available space
 - When entity dies, mark its space available
 - Also save time by not 'deleting' dead entities

ECS Memory Pool

Instead of storing **vectors of Entities**
Let's have a **vector of each Component** type.

Instead of columns (Entities) being contiguous
Change so that rows (components) are contiguous

ECS Mem Pool - Vec<Component>

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1									
C2									
C3									
C4									

Each row is a `Vector<Component>`
Each Entity ID is the index into the vector
Now each Component stored contiguously

Entity as an Index

- The Entity class will **no longer store its components** as member data
- Entity will hold ONLY an **integer index**
- Entity functions will now **access our memory pool** and look up the data at the associated index
- How the hell do we implement this?

Entity Manager

```
1. typedef std::tuple<
2.     std::vector<Component1>,
3.     std::vector<Component2>,
4.     std::vector<Component3>,
5.     std::vector<Component4>
6. > EntityComponentVectorTuple;

7. // to get an Entity's Component type T
8. EntityComponentVectorTuple data;
9. std::get<std::vector<T>>(data)[entityID];
```

EntityMemoryPool

```
1.  class EntityMemoryPool
2.  {
3.      size_t                m_numEntities;
4.      EntityComponentVectorTuple m_pool;
5.      std::vector<std::string>   m_tags;
6.      std::vector<bool>         m_active;
7.      EntityMemoryPool(size_t maxEntities);
8.  public:
9.      static EntityMemoryPool & Instance() {
10.         static EntityMemoryPool pool(MAX_ENTITIES);
11.         return pool;
12.     }
13. }
```


EntityMemoryPool

```
1.  class EntityMemoryPool
2.  {
3.  public:
4.      template <typename T>
5.      T& GetComponent(size_t entityID){
6.          return std::get<std::vector<T>>(m_data)[entityID];
7.      }
8.      const std::string & getTag(size_t entityID) const {
9.          return m_tags[entityID];
10.     }
11. }
```

Entity Class

```
1.  class Entity
2.  {
3.      size_t m_id;
4.  public:
5.      template<typename T>
6.      T & GetComponent() {
7.          return EntityMemoryPool::Instance().GetComponent<T>(m_id);
8.      }
9.      template <typename T>
10.     bool hasComponent() {
11.         return EntityMemoryPool::Instance().hasComponent<T>(m_id);
12.     }
13. }
```

EntityManager Class

```
1. class EntityManager
2. {
3.     std::vector<Entity> m_entities;
4. public:
5.     Entity EntityManager::addEntity(const std::string & tag)
6.     {
7.         Entity e = EntityMemoryPool::Instance().addEntity(tag);
8.         m_entitiesToAdd.push_back(e);
9.         return e;
10.    }
11. }
```

EntityMemoryPool

```
1. class EntityMemoryPool
2. {
3. public:
4.     Entity addEntity(const std::string & tag)
5.     {
6.         size_t index = getNextEntityIndex();
7.         // set all vectors[index] = 0
8.         // set tag[index], active[index] = defaults
9.         return Entity(index);
10.    }
11. }
```

ECS Memory Pool

- When we create the memory pool, we have many copies of a blank default constructed Component of each given type
- Recall: Each Component has a Boolean associated with whether a given Entity actually contains that component
- Entity also has an 'active' boolean

ECS Memory Pool

[illegible]

ECS Memory Pool Indexing

- When we want to put a new Entity into the memory pool, we need to find a location for it within the pool
- Simplest way: scan from beginning to end of pool to find the first location of a non-active entity and use that slot

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	" "	" "	" "	" "	" "	" "	" "	" "
Active	T	F	F	F	F	F	F	F	F

```
Entity p = entityManager.addEntity("P");
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	0, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	" "	" "	" "	" "	" "	" "	" "	" "
Active	T	F	F	F	F	F	F	F	F

```
Entity p = entityManager.addEntity("P");  
p.addComponent<C1>();
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	" "	" "	" "	" "	" "	" "	" "	" "
Active	T	F	F	F	F	F	F	F	F

```
Entity p = entityManager.addEntity("P");  
p.addComponent<C1>();  
p.getComponent<C1>.value = 7;
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
Entity q = entityManager.addEntity("Q");
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	0, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
Entity q = entityManager.addEntity("Q");  
q.addComponent<C2>();
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
Entity q = entityManager.addEntity("Q");  
q.addComponent<C2>();  
q.getComponent<C2>().value = 3;
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
Entity q = entityManager.addEntity("Q");  
q.addComponent<C2>();  
q.getComponent<C2>().value = 3;  
q.removeComponent<C2>();
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	7, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"P"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	F	T	F	F	F	F	F	F	F

p.destroy()

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"R"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
p.destroy()
```

```
Entity r = entityManager.addEntity("R");
```

ECS Memory Pool

	E0	E1	E2	E3	E4	E5	E6	E7	E8
C1	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C2	0, F	3, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
C3	0, T	0, F	0, F	0, F	0, F	0, F	0, F	0, F	0, F
Tag	"R"	"Q"	" "	" "	" "	" "	" "	" "	" "
Active	T	T	F	F	F	F	F	F	F

```
p.destroy()
```

```
Entity r = entityManager.addEntity("R");
```

```
r.addComponent<C3>();
```

ECS Systems

```
// movement
for (auto e : entities)
    e.get<Transform>.pos += e.get<Transform>.velocity

// rendering
for (auto e : entities)
    draw(e.get<Animation>.sprite)

// collisions
for (auto e : entities)
    // e.get<CBoundingBox>
    // e.get<Transform>
```